循环冗余校验

题目背景

数据在传输过程中可能受到干扰，导致某些位出错. 循环冗余校验（Cyclic Redundancy Check，CRC）是一种常用的检错方法，以太网和 ZIP 压缩文件都使用它.

发送方在信息后面接一段校验码，再一起发出. 接收方对收到的内容再做一次除法：余数全为 `0` 时未发现错误，否则说明传输出错：

发送方										传输后										接收方得到的余数			
1	0	1	0	0	1	0	0	1	→	1	0	1	0	0	1	0	0	1	→	0	0	0	未发现错误
1	0	1	0	0	1	0	0	1	→	1	0	1	1	0	1	0	0	1	→	0	1	1	发现错误

绿色是校验码，红色是传错的第 4 位

题目描述

现在由你担任发送方. 给定两个 01 串：信息串 M 和生成串 G. 请在 M 末尾添加 r 个 `0` 得到 X，再用模 2 除法计算 X 除以 G 的商 Q 和余数 R. 余数 R 就是校验码，它的位数 r 为：

$$r = \underline{|G| - 1} \leftarrow \text{生成串的长度减 1}$$

01 串. 只含 `0` 和 `1` 的字符串称为 01 串，$|S|$ 表示字符串 S 的长度.

异或. 两位相同得 `0`，不同得 `1`. 两个等长的 01 串异或，就是逐位异或.

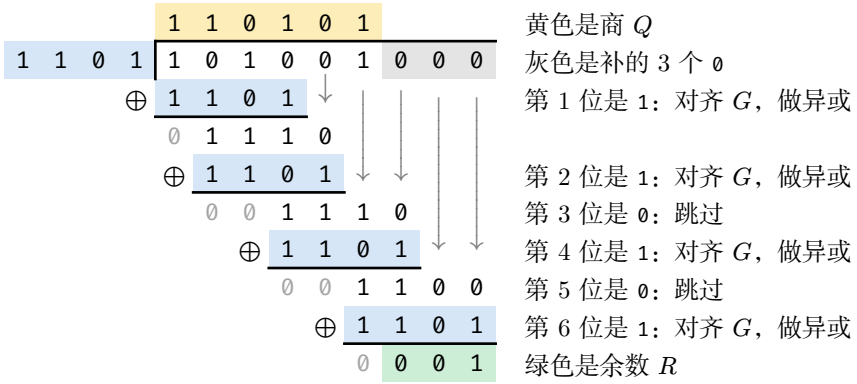
$$0 \oplus 0 = 0 \qquad 0 \oplus 1 = 1$$
$$1 \oplus 0 = 1 \qquad 1 \oplus 1 = 0$$

$$\begin{array}{r} 1\ 1\ 0\ 0 \\ \oplus\ 1\ 0\ 1\ 0 \\ \hline 0\ 1\ 1\ 0 \end{array}$$

模 2 除法. 步骤如下：

1. 从左到右依次看 X 的前 $|M|$ 位. 如果当前位是 `1`，商写 `1`，并把 G 的第一位对齐当前位，用 G 与 X 中对齐的 $|G|$ 位逐位异或，结果写回 X. 如果当前位是 `0`，商写 `0`，X 不变.
2. 写下的 $|M|$ 个商位连起来是商 Q. 此时 X 的最后 r 位是余数 R.

例子. 当 $M =$ `101001`，$G =$ `1101` 时，$r = 3$，除法竖式如下：



输入格式

第一行包含一个整数 T（$1 \le T \le 1000$），表示数据组数.

接下来 T 行，每行包含用一个空格隔开的 01 串 M 和 G（$|M| \ge 1$，$|G| \ge 2$）.

G 的第一个字符是 `1`，M 可以以 `0` 开头或全为 `0`. 所有 $|M|$ 之和与所有 $|G|$ 之和都不超过 5000.

例如样例第一组： `101001`（M） `1101`（G）

输出格式

每组数据输出一行，包含用一个空格隔开的 Q 和 R.

Q 去掉前导 `0`，例如 `0011` 输出 `11`，全为 `0` 时输出一个 `0`. R 恰好输出 r 位，保留前导 `0`.

对应的输出： `110101`（Q） `001`（R）

样例

标准输入	标准输出
5 101001 1101 0011 101 000 1011 1 1001 1011 1011	110101 001 11 11 0 000 1 001 1000 000